

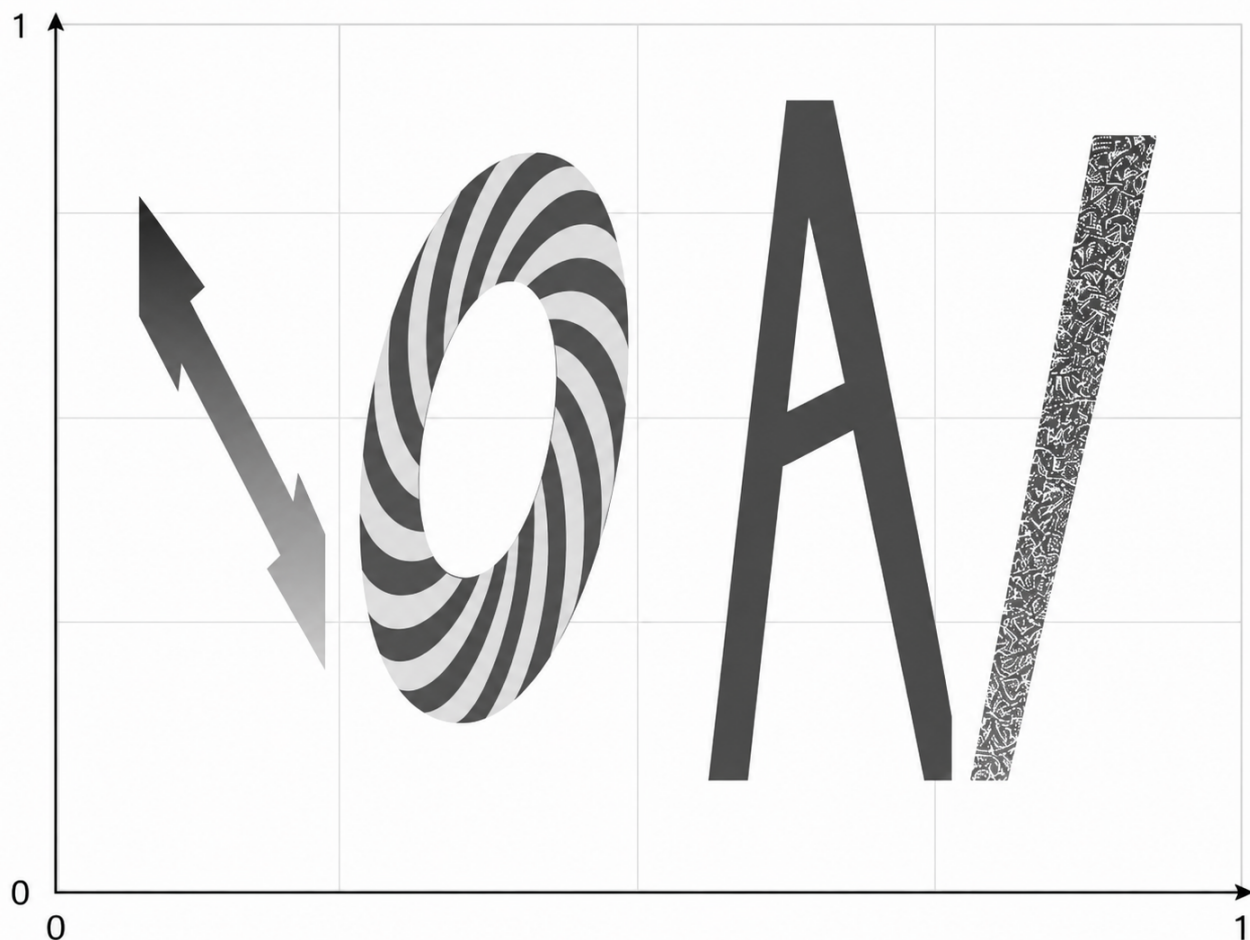
Câmpul IOAI

- **Limită de timp:** 5 minutes
- **Spațiu de stocare:** 5 GB
- **Dimensiunea soluției:** `solution.ipynb`, `custom_model.py` ≤ 1 MB împreună
- **Modele preantrenate:** niciunul — antrenați de la zero, fără internet în momentul evaluării
- **Scor baseline:** 31.2187

Sarcină

Primarul orașului Astana dorește să decoreze orașul cu logouri IOAI stilizate. Fiind statistician, el consideră totul — inclusiv logoul — drept o funcție spațială $F(x, y, \overline{W})$, unde $x, y \in [0, 1]$ reprezintă coordonate într-un plan 2D, iar $\overline{W}$ este un set de parametri ascunși care definesc atribute stilistice precum culorile și unghiurile literelor.

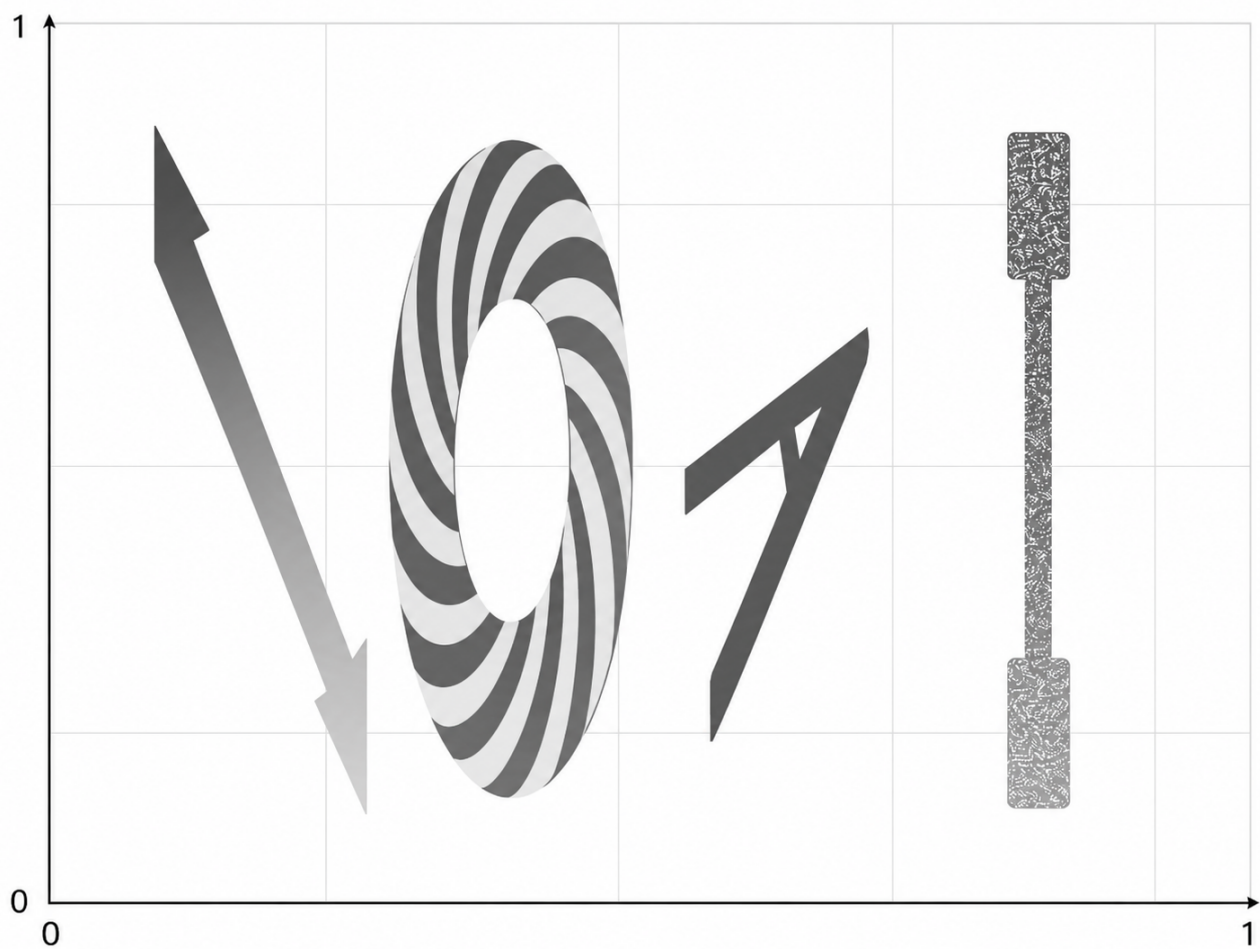
Deoarece F este prea complexă pentru a fi exprimată ca o ecuație matematică explicită, sarcina dumneavoastră este să antrenați o rețea neuronală care să o aproximeze. Rețeaua va produce o valoare a **câmpului IOAI** pentru orice pereche de coordonate (x, y) , generând pe întregul plan o vizualizare completă sub formă de hartă termică a logoului. Iată un exemplu de vizualizare sub formă de hartă termică a lui F cu anumiți parametri ascunși specifici $\overline{W}$.

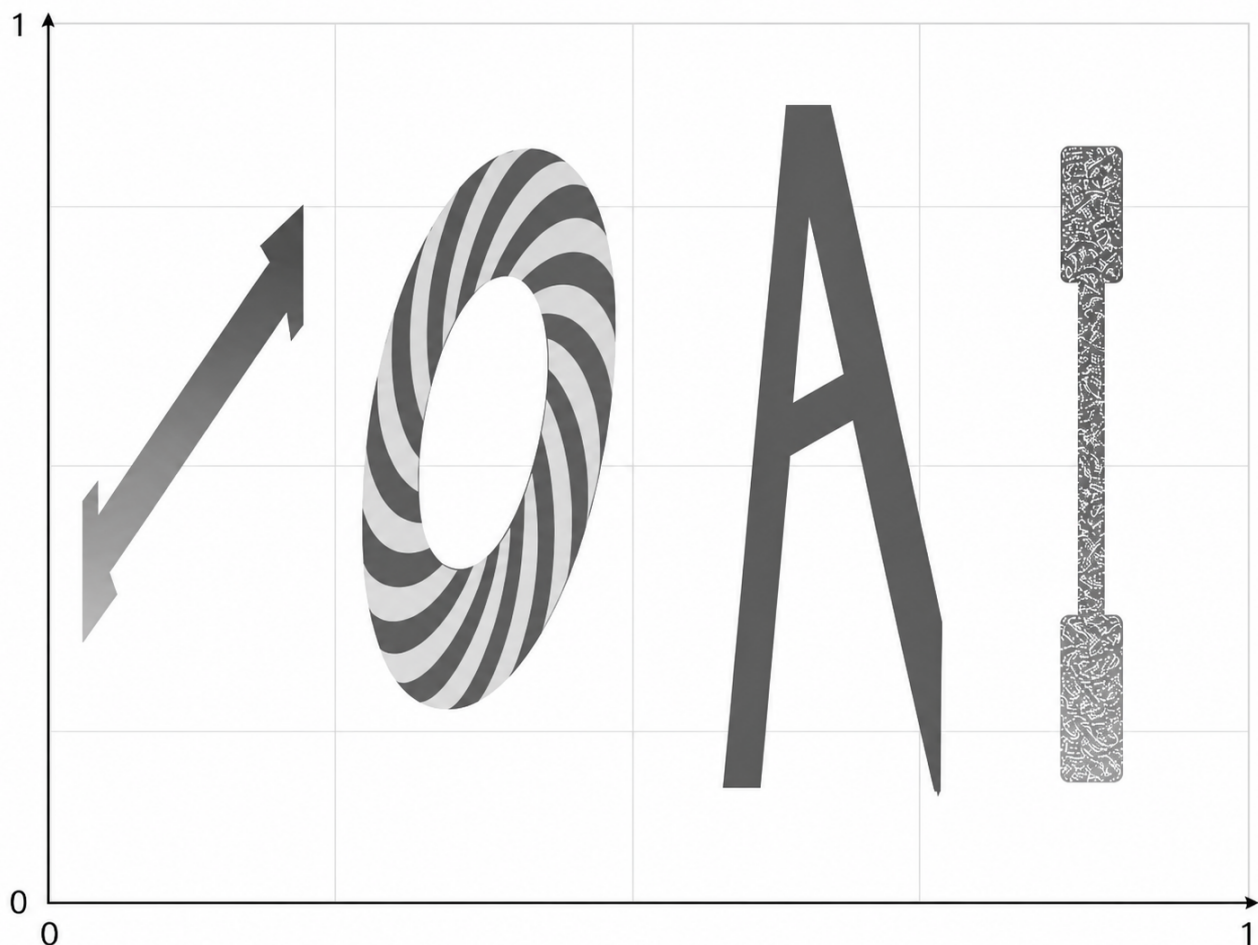


Din ce este alcătuit câmpul IOAI? Din patru litere și fundal.

- Valorile din interiorul primei litere $\mathbb{I}$ sunt foarte mari ($1e+10$ și mai mult), cu un gradient liniar
- Valorile din litera $\circ$ prezintă un model în spirală
- Valoarea din interiorul literei Δ este întotdeauna -1
- Valorile din interiorul ultimei litere $\mathbb{I}$ trebuie să fie valori aleatorii din intervalul $[-2026, 2026]$, chiar dacă sunt evaluate de două ori în același punct
- În exteriorul literelor, valoarea este întotdeauna zero

Funcția are parametri ascunși $\overline{W}$, care influențează scara și înclinarea literelor, precum și intervalul valorilor din interiorul primei litere $\mathbb{I}$. Totuși, literele nu se vor intersecta. Iată câteva exemple ilustrative ale aspectului câmpului IOAI pentru diferite valori ale lui $\overline{W}$:





Ce vi se oferă:

Această problemă NU conține dataseturi. În schimb, vi se oferă funcția generatoare, configurată prin fișierul de configurare JSON de la `data/train_config/field_config.json`.

Configurația de test este ascunsă, dar este de natură similară. Sarcina dumneavoastră este să ajustați modelul pe generatorul furnizat, folosind oricât de multe date doriți. Distribuțiile dumneavoastră „train” și „test” sunt generate de același generator — doar că nu știți în ce puncte (x_i, y_i) veți fi evaluați.

Trimiterea dumneavoastră trebuie să fie alcătuită din:

- clasa modelului de antrenare, salvată ca `custom_model.py`. Acest model trebuie să moștenească din clasa `torch.nn.Module` și să utilizeze numai importuri `torch`. Trebuie să conțină clasa `CustomModel` utilizată în notebook-ul `solution.ipynb`.
- notebook-ul `solution.ipynb`, care va produce ponderile `model.pt`

Punctaj

Pentru fiecare regiune, scorul minim este 0, iar scorul maxim este 1. Scorul final este calculat ca media pentru toate cele cinci regiuni (câte una pentru fiecare dintre cele patru litere și una pentru

fundal) și înmulțit cu 100. Există o **penalizare pentru numărul de parametri**:

Dacă modelul dumneavoastră are mai mult de 20260 de parametri, scorul este înjumătățit.

Numărul de parametri este măsurat prin `sum(p.numel() for p in model.parameters())`. Ne așteptăm ca modelul dumneavoastră să funcționeze și în mod stocastic, având componenta `PyTorch nn.Dropout` ca parte a modelului.

Pentru regiunile standard

Pentru fiecare regiune R (prima literă `I`, `O`, `A`, `Background`), evaluăm modelul pe $N_R = 512$ puncte de test (x_i, y_i) , cu valorile reale v_i și predicțiile $\hat{v}_i$. Folosim eroarea absolută medie normalizată (MAE) drept metrică principală. MAE este definită astfel:

$$\text{MAE}(R) = \frac{1}{N_R} \sum_{i=1}^{N_R} |\hat{v}_i - v_i|.$$

Iar normalizarea este efectuată astfel:

$$\text{score}(R) = 1 - \min \left(\frac{\text{MAE}(R)}{s_R}, 1 \right),$$

unde $s_R > 0$ este o constantă de scalare.

Pentru regiunea ultimei litere `I`

În această regiune, **dropout este activat în timpul evaluării**. Pentru fiecare punct de test j :

1. Rulăm modelul de $K = 10$ ori pentru a obține $\hat{v}_{j,1}, \dots, \hat{v}_{j,K}$.
2. Dacă oricare dintre ieșiri se află în afara intervalului $[-2026, 2026]$, atunci $\text{pointScore}(j) = 0$.
3. În caz contrar, calculăm abaterea standard σ_j a celor K ieșiri și o convertim într-un scor:

$$\text{pointScore}(j) = \min \left(\frac{\sigma_j}{s_E}, 1 \right),$$

unde $s_E > 0$ este o constantă de scalare fixă.

Scorul regiunii este media pentru toate punctele din regiune:

$$\text{score}(I_{last}) = \frac{1}{N_E} \sum_{j=1}^{N_E} \text{pointScore}(j),$$

unde $N_E = K * N_R$.

În termeni simpli, cu cât aveți mai multă diversitate, cu atât scorul dumneavoastră pentru această regiune va fi mai mare. **Nu puteți utiliza aleatorietatea în formă pură, inclusiv funcțiile**

PyTorch `rand*` și `_uniform`; aleatorietatea trebuie să provină din inferența cu dropout activat.

Cum să trimiteți soluția

1. Deschideți `solution.ipynb` și rulați toate celulele.
2. Îmbunătățiți modelul `CustomModel` din `custom_model.py`
3. Asigurați-vă că ultima celulă salvează modelul dumneavoastră în fișierul `model.pt`.
4. În fila Git din JupyterLab, adăugați în staging, introduceți un comentariu și efectuați commit pentru `solution.ipynb` și `custom_model.py`, apoi efectuați push.
5. Reveniți la pagina concursului și apăsați pe **Submit**. Comentariul trimiterii trebuie să fie același cu cel de la pasul anterior.